Calorica

Приложение для учёта питания, калорий и БЖУ

Этап 3 · архитектура решения



Технологии решения

Один репозиторий, независимые сборки и выпуски

Android	Kotlin / Compose
Сервер	Kotlin / Spring Boot
База данных	PostgreSQL
Доступ к данным	Spring JDBC / SQL
Миграции	Liquibase
Выпуск	Actions / Docker

Серверная архитектура

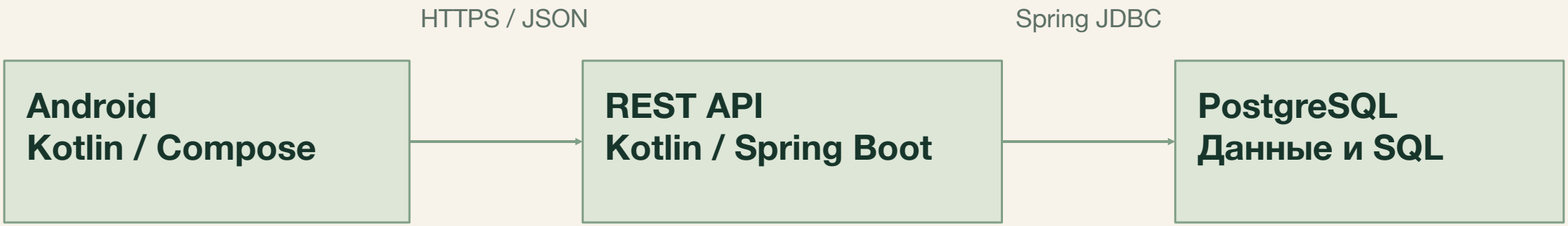
Один сервер

Модульный монолит.
Одна база данных.
Общие транзакции.

REST / JSON / OpenAPI.
Расчёты на сервере.

Архитектура приложения

Клиент показывает данные. Сервер проверяет правила и считает КБЖУ.



Формы и состояние
Запросы и результат

Сессии и права
Сценарии и транзакции

Нормализованная БД
Рекурсивный расчёт

Карточка, дневник и сводка используют один серверный механизм расчёта.

Модули серверной части

Один процесс Spring Boot, предметные области разделены пакетами.

Аккаунт и каталог

Модуль	Ответственность
accounts	Аккаунты и сессии
profile	Личные ориентиры
catalog	Продукты и составы

Питание и инфраструктура

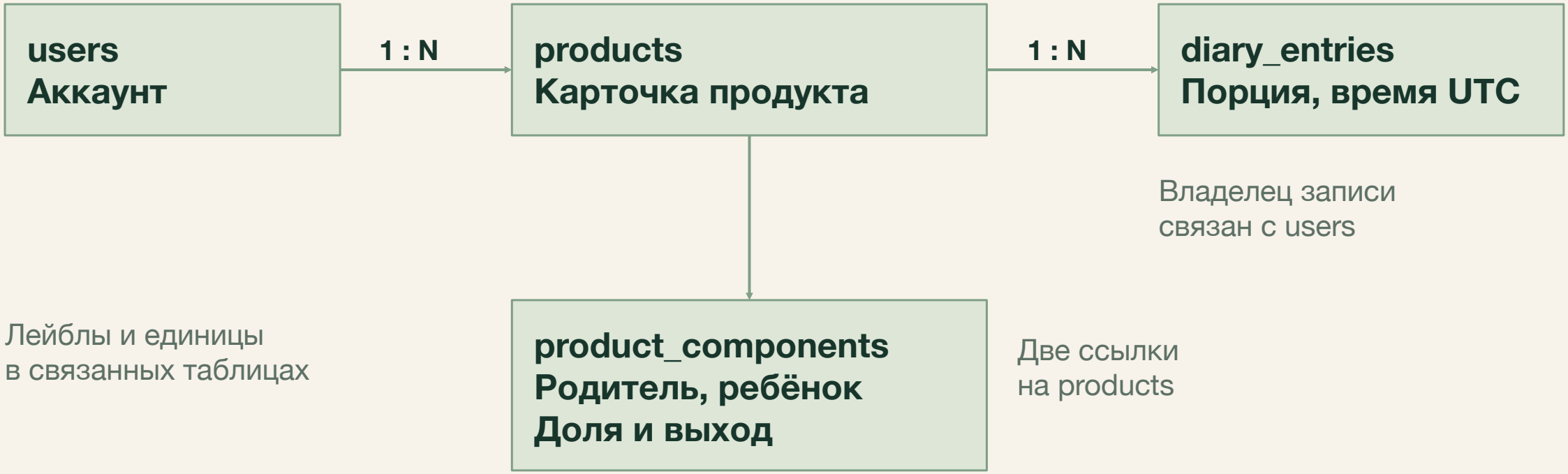
Модуль	Ответственность
diary	Записи питания
analytics	Календарные сводки
platform	Общие средства

API, сценарии, домен и SQL разделены внутри каждого модуля.

Сценарий управляет транзакцией. Чужие таблицы модуль не изменяет.

Модель данных

Продукт, связь состава и факт питания хранятся отдельно.



Без снимка КБЖУ. Мягкое удаление карточки сохраняет историю питания.

Сохранение и расчёт

Проверки при записи и вычисления при чтении решают разные задачи.

Сохранение состава

1. Блокировка владельца
2. Права, доли и отсутствие цикла
3. **Одна транзакция и commit**

Чтение КБЖУ

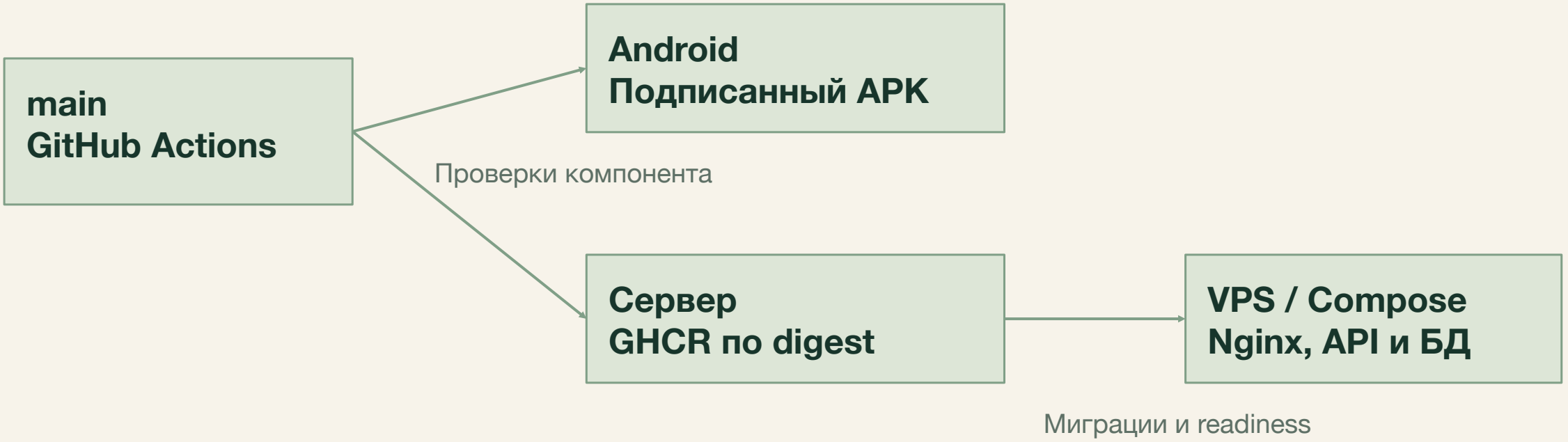
1. WITH RECURSIVE до листьев
2. Каждый путь графа даёт вклад
3. **Сумма с учётом выхода блюда**

Массы рецепта превращаются в доли. Съеденная масса хранится в дневнике.

Безопасность и выпуск

HTTPS, серверные права и независимые workflows компонентов

Access JWT и refresh с ротацией. Argon2id. Защита Keystore в Android.



Копии БД и проверка восстановления. Перенос VPS через конфигурацию.

Итоги третьего этапа

Требования второго этапа дополнены проектированием решения.

01	**Технологии**	Kotlin, Compose, Spring Boot, JDBC, PostgreSQL и Liquibase.
02	**Архитектура**	Границы компонентов, модули сервера и модель данных.
03	**Проверки**	Расчёты, циклы, доступ, миграции и восстановление.

Объём семестрового MVP сохранён.
Каждое ключевое решение связано с проверкой.